

PANDAS CORE API - QUICK / BASICS / SERIES / MISSING

CONCEPT NAME -> syntax only | packed combined sheet

MISSING DATA <pre>pd.isnull(x) pd.notnull(x) pd.isna(x) pd.notna(x) df.isnull().sum() df.fillna(0) df.fill() df.bfill() df.interpolate() df.dropna() df.dropna(how='all') df.dropna(thresh=N) df['col'].sum(skipna=True)</pre>	INSPECTION <pre>df.head() df.head(n=10) df.tail() df.tail(n=1) df.shape df.shape[0] df.shape[1] df.columns df.dtypes df.info() df.describe() type(df)</pre>	SERIES METHODS <pre>first_row.keys() s.mean() s.median() s.mode() s.min() s.max() s.std() s.var() s.quantile(q) s.describe() s.sum() s.count()</pre>	DATETIME COLUMNS <pre>scientists['born_dt'] = pd.to_datetime(scientists['Born'], format='%Y-%m-%d') scientists['died_dt'] = pd.to_datetime(scientists['Died'], format='%Y-%m-%d') scientists['age_days'] = scientists['died_dt'] - scientists['born_dt'] scientists['age_years'] = scientists['age_days'].dt.days // 365</pre>	SERIES TRANSFORM <pre>s.isin([...]) s.apply(func) s.map(func_or_dict) s.astype(dtype) s.replace(a, b) s.sample(n) s.to_frame() s corr(other) s.cov(other) s.equals(other)</pre>
BOOLEAN SUBSETTING <pre>ages > ages.mean() ages[ages > ages.mean()] ages[mask] scientists[scientists['Age'] > scientists['Age'].mean()] scientists[(scientists['Age'] > 40) & (scientists['Occupation'] == 'Chemist')] scientists.query('Age > 40 and Occupation == "Chemist"')</pre>	SUBSETTING <pre>df['col'] df[['col1', 'col2']] df.iloc[:, n] df.loc[label] df.iloc[pos] df.loc[row_sel, col_sel] df[df['col'] > x] df[(cond1) & (cond2)] df.query('cond1 and cond2')</pre>	TIDY & RESHAPING <pre>pd.melt(df, id_vars=['id'], var_name='var', value_name='val') df.pivot_table(index='id', columns='var', values='val').reset_index() df['col'].str.split('_', expand=True) df[['a', 'b']].drop_duplicates() df.merge(sub, on='a', b') glob.glob('data/*.csv') pd.concat([pd.read_csv(f) for f in files])</pre>	NAN CHECKS <pre>pd.isnull(nan) pd.isnull(42) pd.notnull(nan) pd.notnull(42) pd.isna(x) pd.notna(x) df.isna() df.notna() df.isnull() df.notnull()</pre>	LOAD & INSPECT <pre>df = pd.read_csv('data.csv') df = pd.read_csv('data.tsv', sep='\t') df.head() df.tail() df.shape df.columns df.dtypes df.info() df.describe()</pre>
READ CSV NA <pre>pd.read_csv('survey_visited.csv') pd.read_csv('survey_visited.csv', keep_default_na=False) pd.read_csv('survey_visited.csv', na_values=[''], keep_default_na=False) pd.read_csv('file.csv', na_values=[99]) keep_default_na=False na_filter=False</pre>	ROW SELECTION <pre>df.loc[0] df.loc[[0, 99, 999]] df.iloc[0] df.iloc[-1] df.iloc[[0, 99, 999]] df.iloc[0:3] df[start:stop:step] df[boo_series]</pre>	EXPORT / IMPORT <pre>df.to_pickle('scientists_df.pickle') pd.read_pickle('scientists_df.pickle') df.to_csv('scientists.csv', index=False) pd.read_csv('scientists.csv') df.to_excel('scientists.xlsx', index=False) pd.read_excel('scientists.xlsx') df.to_parquet('scientists.parquet', index=False) pd.read_parquet('scientists.parquet')</pre>	EXPORT / IMPORT <pre>df.to_csv('f.csv', index=False) pd.read_csv('f.csv') df.to_excel('f.xlsx', index=False) pd.read_excel('f.xlsx') df.to_pickle('f.pickle') pd.read_pickle('f.pickle') df.to_parquet('f.parquet') pd.read_parquet('f.parquet')</pre>	MODIFY DATAFRAME <pre>df['new_col'] = ... sorted_ages = ages.sort_values() scientists['Age'] = scientists['Age'].sample(frac=1).reset_index(drop=True) df.rename(columns={'old_name': 'new_name'}) df.drop(columns=['col_to_remove']) df[['col_b', 'col_a']] df['col'].apply(lambda x: x * 2) df.apply(lambda row: row['a'] + row['b'], axis=1)</pre>
COMBINING DATA <pre>pd.concat([df1, df2]) pd.concat([df1, df2], axis=1) pd.concat([df1, df2], ignore_index=True) pd.concat([df1, df2], join='inner') left.merge(right, on='key') left.merge(right, left_on='a', right_on='b', how='outer')</pre>	PLOTTING <pre>df['col'].plot.hist() df.plot.scatter(x='a', y='b') sns.histplot(df['col']) sns.scatterplot(x='a', y='b', data=df) sns.boxplot(x='cat', y='num', data=df, hue='cat2') sns.lmplot(x='a', y='b', data=df, col='facet_col') fig, ax = plt.subplots()</pre>	CREATING & MODIFYING <pre>pd.Series([1, 2, 3], index=['a', 'b', 'c']) pd.DataFrame({'col1': [...], 'col2': [...]) df['new_col'] = ... df.rename(columns={'old': 'new'}) df.drop(columns=['col']) df['col'].apply(func) df.apply(lambda row: ..., axis=1)</pre>	GROUPBY / AGGREGATION <pre>df.groupby('col')['target'].mean() df.groupby(['col1', 'col2'])[['t1', 't2']].mean() df.groupby('col').agg({'t1': 'mean', 't2': 'sum'}) df.groupby('col').agg(avg=('t1', 'mean'), total=('t2', 'sum')) df['col'].nunique() df['col'].value_counts()</pre>	COUNT MISSING <pre>ebola.shape ebola.count() ebola.shape[0] - ebola.count() np.count_nonzero(ebola.isnull()) np.count_nonzero(ebola['Cases_Guinea'].isnull()) ebola['Cases_Guinea'].value_counts(dropna=False).head() df.isnull().sum() df.isna().sum()</pre>
DO NOT USE <pre>df.shape() df[0] df[[0]] df.loc[-1] df['a', 'b'] .ix[]</pre>	DEPRECATED / REMOVED <pre>.ix[...] -> .loc[...] / .iloc[...] df.append(...) -> pd.concat([...]) sns.distplot(...) -> sns.histplot() / sns.kdeplot() df.fillna(method='ffill') -> df.ffill() df.fillna(method='bfill') -> df.bfill() inplace=True -> df = df.method(...)</pre>	COLUMN SELECTION <pre>df['country'] df.country df[['country', 'continent', 'year']] df.iloc[:, 0] df.iloc[:, [0, -1]] df.iloc[:, 0:3]</pre>	CALCULATIONS <pre>ebola.sum() ebola.sum(skipna=True) ebola.sum(skipna=False) ebola.mean() ebola.count() ebola.min() ebola.max()</pre>	AGGREGATION <pre>df.groupby('continent').agg({'lifeExp': 'mean', 'pop': 'sum'}) df.groupby('continent')['lifeExp'].agg(['mean', 'min', 'max', 'count']) df.groupby('continent').agg(avg_life_exp=('lifeExp', 'mean'), total_pop=('pop', 'sum'))</pre>
DO NOT USE <pre>nan == nan x == np.nan df.fillna(method='ffill') df.fillna(method='bfill') inplace=True .ix[]</pre>	DTYPES <pre>object int64 float64 bool datetime64[ns] category</pre>	SERIES UNIQUE / SORT <pre>s.unique() s.nunique() s.value_counts() s.sort_values() s.sort_index() s.drop_duplicates()</pre>	SERIES ATTRIBUTES <pre>first_row.index first_row.values first_row.dtype first_row.shape first_row.size first_row.T</pre>	FILL MISSING <pre>ebola.fillna(0) ebola = ebola.fillna(0) ebola.ffill() ebola.bfill() ebola.interpolate() ebola.interpolate(method=...)</pre>
IMPORTS <pre>import pandas as pd import numpy as np import matplotlib.pyplot as plt import seaborn as sns</pre>	ROWS + COLUMNS <pre>df.loc[42, 'country'] df.iloc[42, 0] df.loc[[0, 99], ['country', 'lifeExp']] df.iloc[[0, 99], [0, 3]] df.loc[row_sel, col_sel]</pre>	GROUPBY <pre>df.groupby('year') df.groupby('year')['lifeExp'] pd.Series(['Wes McKinney', 'Creator of Pandas'], index=['Person', 'Who']) pd.Series({'goat': 4, 'amoeba': np.nan}) pd.Series(5, index=['a', 'b', 'c'])</pre>	CREATE SERIES <pre>pd.Series(['banana', 42]) pd.Series(['Wes McKinney', 'Creator of Pandas'], index=['Person', 'Who']) pd.Series({'goat': 4, 'amoeba': np.nan}) pd.Series(5, index=['a', 'b', 'c'])</pre>	DROP MISSING <pre>ebola.dropna() ebola.dropna(how='any') ebola.dropna(how='all') ebola.dropna(thresh=10) ebola.dropna().shape</pre>
COUNTING <pre>df['country'].nunique() df.groupby('continent')['country'].nunique() df['continent'].value_counts() df.groupby('continent')['country'].count()</pre>	PANDAS OBJECTS <pre>pd.Series(...) pd.DataFrame(...) type(df['country']) type(df[['country']])</pre>	QUICK PLOT <pre>import matplotlib.pyplot as plt yearly_life_exp = df.groupby('year')['lifeExp'].mean() yearly_life_exp.plot() plt.show()</pre>	CREATE DATAFRAME <pre>pd.DataFrame({'Name': [...], 'Age': [...]) pd.DataFrame(data=..., index=..., columns=...) pd.DataFrame([['n1', 'n2', 'n3', 'n4']], columns=['A', 'B', 'C', 'D'])</pre>	VECTORIZED OPERATIONS <pre>ages + ages ages * 2 pd.Series([1, 2, 3]) + pd.Series([10, 20]) ages.values + np.array([...])</pre>
DATA LOADING <pre>df = pd.read_csv('data.csv') df = pd.read_csv('data.tsv', sep='\t') df = pd.DataFrame(...)</pre>	IMPORTS <pre>import numpy as np import pandas as pd from numpy import nan</pre>	REINDEX NAN <pre>life_exp = gapminder.groupby('year')['lifeExp'].mean() y2000 = life_exp[life_exp.index > 2000] y2000.reindex(range(2000, 2010))</pre>	IMPORTS <pre>import pandas as pd import numpy as np</pre>	& <pre> ~</pre>
MERGE NAN <pre>visited.merge(survey, left_on='ident', right_on='taken')</pre>	IMPORT <pre>import pandas as pd</pre>		DO NOT USE <pre>and / or / not on Series inplace=True</pre>	CREATE NAN <pre>pd.Series({'goat': 4, 'amoeba': np.nan}) scientists['missing'] = np.nan</pre>